

Catalogizer

Gestion avancée de collections multimédias multi-protocoles — détectez, cataloguez et enrichissez tout ce que vous possédez.

Résumé

Catalogizer est un système auto-hébergé de gestion de collections multimédias qui détecte, catégorise et organise automatiquement les médias sur SMB, FTP, NFS, WebDAV et les systèmes de fichiers locaux, avec une surveillance en temps réel, un stockage chiffré, un enrichissement des métadonnées externes et une interface moderne React, soutenue par un moteur haute performance Go API.

Description courte

Un gestionnaire de médiathèque multi-protocoles de niveau professionnel. Un moteur Go/Gin REST API détecte plus de 50 types de médias sur des sources SMB/FTP/NFS/WebDAV/locales, les enrichit via TMDB/IMDb/MusicBrainz/Steam et bien d'autres, et propose une application web React en temps réel sur une base de données chiffrée SQLCipher.

Description détaillée

La plupart des gestionnaires de médias exigent d'abord que vous fassiez des concessions : tout regrouper sur un seul disque, dans un seul format, d'un seul type, et seulement alors ils vous viennent en aide. Catalogizer part d'un postulat inverse — votre collection existe déjà là où elle se trouve, dispersée sur des partages NAS et des protocoles qui ne s'entendront jamais — et s'y adapte. Il parle les protocoles déjà utilisés par le stockage — SMB/CIFS, FTP/FTPS, NFS, WebDAV et système de fichiers local — derrière une abstraction client unifiée, de sorte qu'un partage Windows, une archive FTP et un montage WebDAV apparaissent identiques aux couches supérieures et peuvent être mélangés, échangés ou retirés sans toucher au code applicatif. Un backend

Go (Gin REST API) surveille en continu ces sources, détecte et classe plus de 50 types de médias (films, séries, musique, jeux, logiciels, documentaires) à mesure que les fichiers apparaissent, et enrichit chaque élément à partir d'une pile de fournisseurs externes — TMDb, IMDb, TVDB, MusicBrainz, Spotify, Steam et bien d'autres — transformant un simple nom de fichier en une entrée de catalogue complète, avec visuels, distribution et métadonnées. Les résultats sont diffusés vers un frontend TypeScript React via WebSockets, de sorte que la bibliothèque se met à jour en direct pendant l'ingestion, sans nécessiter de rafraîchissement manuel, et chaque octet de métadonnées est conservé dans une base de données chiffrée SQLCipher, protégée par une authentification basée sur JWT et des rôles.

Là où la plupart des catalogues s'effondrent dès qu'un partage tombe, Catalogizer est conçu pour rester opérationnel malgré les pannes. Une défaillance temporaire SMB est absorbée par une reconnexion avec backoff exponentiel, un disjoncteur empêchant de solliciter un hôte hors service, une surveillance continue de l'état de santé et un cache de métadonnées hors ligne qui continue de répondre aux requêtes utilisateur à partir du dernier état valide — la différence entre « *l'application entière est hors service parce qu'un NAS a redémarré* » et « *une source est dégradée, mais tout le reste fonctionne* ». Au-delà du catalogage, il sert aussi d'outil opérationnel pour une collection : analyses des tendances de croissance et suivi de la qualité/des versions, génération de rapports professionnels PDF, service de conversion PDF vers image/texte/HTML, export/import des favoris (JSON/CSV), et synchronisation cloud vers S3, Google Cloud Storage ou des dossiers locaux. Et ce n'est pas un monolithe qui se trouve être volumineux — il est délibérément assemblé à partir de 21 sous-modules Go `digital.vasic.*` réutilisables, ainsi que de paquets clients TypeScript, chacun testé et versionné indépendamment, de sorte que les mêmes composants robustes d'authentification, de système de fichiers, de streaming et d'observabilité qui font fonctionner Catalogizer alimentent aussi l'ensemble de la famille de produits. La qualité n'est pas auto-déclarée : le framework Challenges et HelixQA soumettent chaque fonctionnalité annoncée à une vérification anti-bluffer, basée sur des preuves.

Pourquoi nous l'avons créé

Les gestionnaires de médias existants partent du principe d'un seul backend de stockage et d'un seul type de média. Les collections réelles s'étendent sur plusieurs partages NAS et protocoles, se dégradent lorsqu'un partage tombe en panne, et mêlent films, musique, jeux et logiciels. Catalogizer a été conçu

pour traiter tous les protocoles sur un pied d'égalité, résister aux aléas du stockage en réseau et offrir un catalogue unique, enrichi et chiffré couvrant l'ensemble des données.

Pourquoi c'est révolutionnaire

Il condense en un seul paquet auto-hébergeable et chiffré ce qui nécessite habituellement une pile d'outils distincts : une ingestion agnostique des protocoles qui traite chaque backend de stockage de manière équivalente, une résilience permettant au catalogue de rester opérationnel malgré les pannes de stockage au lieu de s'effondrer avec elles, et un enrichissement multi-fournisseurs sophistiqué qui transforme des fichiers bruts en une bibliothèque navigable et documentée. Le bénéfice de l'architecture modulaire est cumulatif : une correction de robustesse pour le client du système de fichiers ou un nouveau plugin de fournisseur n'est appliquée qu'une seule fois et profite à tous les utilisateurs, de sorte que Catalogizer s'améliore constamment au rythme de l'écosystème qui l'entoure. En somme, c'est la différence entre un simple index de médias et un *système* de médias – un système que vous possédez, qui survit aux infrastructures instables, et dont les rouages sont éprouvés plutôt que simplement promis.

Ce qui est innovant

- Client unifié de système de fichiers multi-protocoles (SMB/FTP/NFS/WebDAV/local) derrière une seule interface.
- Cache hors ligne + disjoncteur pour maintenir l'utilisation du catalogue pendant les pannes de stockage.
- Extraction complète en 21 sous-modules `digital.vasic.*` Go et modules clients TS réutilisables.
- Catalogue chiffré au repos (SQLCipher) avec synchronisation WebSocket en temps réel vers l'interface utilisateur.
- Assurance qualité fondée sur des preuves via le framework Challenges et l'intégration HelixQA.

Défis et solutions

- **Stockage réseau instable** : résolu grâce à un repli exponentiel, un disjoncteur, des vérifications d'état et un cache hors ligne à politique d'éviction qui fournit des métadonnées en cache lorsque les sources sont inaccessibles.
- **Hétérogénéité des protocoles** : résolue en abstraisant chaque protocole derrière un client commun `digital.vasic.filesystem`, rendant les couches supérieures indépendantes du protocole.

- **Sécurité des données** : résolue avec le chiffrement SQLCipher au repos, ainsi que l'authentification JWT/RBAC et un middleware de nettoyage des requêtes.
- **Maintenabilité à grande échelle** : résolue en extrayant toute la logique générique dans des sous-modules testés indépendamment plutôt que dans un monolithe.

Pile technologique (pourquoi et comment)

- **Go + Gin** — noyau REST API haute performance (catalog-api) ; choisi pour sa concurrence et son débit sur les charges de travail de surveillance continue.
- **TypeScript + React + Tailwind (Vite)** — interface utilisateur catalog-web réactive avec mises à jour en temps réel.
- **WebSockets** — synchronisation en direct des données entre le hub backend et l'interface utilisateur.
- **SQLCipher (SQLite chiffré)** — stockage de métadonnées chiffré au repos ; double support SQLite/PostgreSQL via `digital.vasic.database`.
- **Clients SMB/FTP/NFS/WebDAV** — ingestion multi-protocoles via `digital.vasic.filesystem`.
- **API de métadonnées externes (TMDB, IMDB, TVDB, MusicBrainz, Spotify, Steam)** — plugins de fournisseurs pour l'enrichissement.
- **Prometheus + OpenTelemetry** — métriques/traçage via `digital.vasic.observability`.
- **Docker / conteneur de construction** — builds reproductibles (Tauri/Rust acheminés via `catalogizer-builder`).
- **Redis** — mise en cache et limitation de débit via `digital.vasic.cache / ratelimiter`.
- **S3 / Stockage Cloud Google** — synchronisation cloud et stockage des points de contrôle.